Lab02 - Stanford Bunny GPU Instancing Benchmark

Controlled comparison of repeated draw calls and GPU instancing

Repository: ComputerGraphicsLabs
Lab: Lab02_BunnyGrid_Instancing
Baseline: OpenGL 3.3 Core Profile, Visual Studio x64-Debug, VSync off

1. Purpose

Lab02 is the controlled follow-up to Lab01. Lab01 renders each Stanford Bunny with a separate **glDrawElements()** call. Lab02 keeps the same mesh, object counts, fixed grid, scale and simple shader workload, but replaces repeated submission with one **glDrawElementsInstanced()** call.

The research question is whether reducing N draw calls to one instanced draw materially reduces frame time, and whether the result differs between the integrated AMD GPU and the NVIDIA RTX 4050 Laptop GPU.

2. Environment

Component	Configuration
API	OpenGL 3.3 Core Profile
Language	C++ / GLSL 3.30
Libraries	GLFW + GLAD
Build	Visual Studio x64-Debug
Model	Stanford Bunny, indexed triangle mesh
Discrete GPU	NVIDIA GeForce RTX 4050 Laptop GPU
NVIDIA driver	617.42
Integrated GPU	AMD Radeon(TM) Graphics
VSync	Off - glfwSwapInterval(0)

3. Implementation

The CPU submits the complete instance count once with **glDrawElementsInstanced(..., bunnyCount)**. The vertex shader uses **gl_InstanceID** to derive row and column coordinates and computes the per-instance x/y offset. All instances reuse the same mesh buffers.

The console reports **offsetLocation = -1**. This is expected because the old uOffset uniform is no longer used and is optimized away.

4. Lab02 Measurements

Approximate steady-state readings were taken after startup transients. The four workloads were 1, 100, 1,000 and 10,000 bunnies.

Bunnies	RTX FPS	RTX ms	AMD FPS	AMD ms
1	~2,250	~0.44	~4,700	~0.21
100	~2,050-2,100	~0.48	~1,400	~0.71
1,000	~727	~1.38	~355	~2.82
10,000	~85.7	~11.67	~81.5	~12.27

5. Lab01 vs Lab02

GPU / Bunnies	Lab01 FPS	Lab02 FPS	Lab01 ms	Lab02 ms
RTX / 1	~2,250	~2,250	~0.44	~0.44
RTX / 100	~708	~2,050-2,100	~1.41	~0.48
RTX / 1,000	~710	~727	~1.41	~1.38
RTX / 10,000	~85	~85.7	~11.7	~11.67
AMD / 1	~4,800	~4,700	~0.21	~0.21
AMD / 100	~1,400	~1,400	~0.71	~0.71
AMD / 1,000	~385	~355	~2.60	~2.82
AMD / 10,000	~83	~81.5	~12.0	~12.27

6. Observations

RTX, 100 bunnies. This is the clearest instancing gain. Frame time falls from about 1.41 ms to about 0.48 ms, roughly a **2.9x reduction**.

RTX, 1,000 bunnies. The implementations converge: about 710 FPS versus 727 FPS.

RTX, 10,000 bunnies. The results are effectively tied at about 85 FPS. The drastic reduction in draw-call count does not materially reduce total frame time here, indicating that draw submission is no longer the dominant cost.

AMD iGPU. Lab01 and Lab02 remain approximately equal across the tested workloads. Small differences should not be treated as reliable wins without repeated statistical runs.

7. Interpretation

The experiment demonstrates that **fewer draw calls do not automatically imply higher FPS**. Instancing attacks CPU/driver submission overhead, but it does not remove the geometry processing required for the instances. Its visible benefit therefore depends on whether draw submission is a significant bottleneck for the current workload.

The 10,000-bunny result is especially useful: radically different submission strategies converge to almost the same frame time. FPS alone cannot identify the exact limiting stage; a profiler would be required for a stronger bottleneck diagnosis.

8. Fixed-Grid and Visibility Limitation

Lab02 deliberately preserves the Lab01 fixed 10-column grid and uScale = 0.10 so that submission strategy remains the primary changed variable. At high counts, the grid extends far outside the viewport. Instancing still requests all instances; clipping is not object-level culling.

9. Main Conclusion

Lab02 successfully answers the question left open by Lab01. On the RTX 4050, instancing produces a strong gain at 100 bunnies but little or no gain at 1,000 and 10,000. On the AMD iGPU, no clear gain appears in the current measurements.

The precise conclusion is: **GPU instancing reduces draw-submission overhead, and the observable performance benefit depends on whether that overhead is a significant bottleneck in the tested workload.**

10. Methodology Notes

These are exploratory measurements from stabilized console readings, not a formal GPU microbenchmark. A stricter follow-up should use Release builds, identical power conditions, repeated timed runs, averages/medians, percentile frame times and CPU/GPU profiling.

11. Next Experiments

Experiment	Goal
Adaptive grid + bounding box	Derive cell size and safe scale from actual mesh bounds.
Frustum culling	Avoid rendering objects outside the view volume.
Spatial hierarchy	Use a quadtree/octree or related hierarchy for scalable visibility tests.
Level of Detail	Reduce geometry when objects occupy few screen pixels.
GPU / compute culling	Move more visibility decisions toward GPU-driven rendering.
Release + profiling	Separate CPU submission, driver and GPU costs more rigorously.

Repository Progression

Lab00: basic Stanford Bunny rendering and portability.
Lab01: repeated draw-call benchmark and performance baseline.
Lab02: GPU instancing and controlled comparison against Lab01.
Next: bounding volumes / adaptive layout, followed by visibility and culling experiments.